整体先给个直观版

V8 的 GC = “分代 + 不同算法组合” 的一整套系统：

- 内存被分成“新生代”和“老生代”两大块（分代 GC）。
- 新生代（刚创建的对象）→ 用快速的 Minor GC（复制算法为主）。
- 老生代（活很久的对象）→ 用标记-清除 + 标记-压缩 的 Major GC。
- 外加：**增量 / 并行 / 空闲时间 GC** 来减少页面卡顿。

下面稍微系统一点说。

---

1. V8 的堆布局（Heap）

V8 在堆上大致分几块空间，最关键的是两类：

- 新生代（New Space）
  - 体积小，只存放“刚创建不久”的对象。
  - 回收非常频繁，但每次都很快。
- 老生代（Old Space）
  - 存活时间较长、从新生代“晋升”上来的对象。
  - 回收频率低，但每次更重一些（算法复杂）。

有点像：

“新生代 = 垃圾堆旁边的临时垃圾桶，老生代 = 垃圾站仓库”。

---

2. 新生代：Minor GC（Scavenge / 复制算法）

目标：快速清掉“短命对象”。

- 新生代通常又分成两块等大小的半区：`From Space` 和 `To Space`。
- 执行 Minor GC 时：
  a. 根对象出发，**标记出新生代中仍然存活的对象**。
  b. 把这些存活对象**复制到** `To Space`，并进行紧凑排列（没有碎片）。
  c. 清空原来的 `From Space`。
  d. 交换两个空间的角色（`To` → `From`，`From` → `To`）。
- **好处：**
  - 死对象一般占大多数，不用管，直接丢掉。
  - 活的对象被复制过去时顺便完成“压缩”，没有碎片，分配新对象只要指针往前挪。

---

3. 对象“晋升”到老生代

不是所有新对象都一直待在新生代：

- 对象如果**经历了多次 Minor GC 还活着**，就会被**晋升到老生代**。
- 如果某个对象**特别大**，V8 也可能直接分配在老生代（避免频繁复制）。

这体现了经典的**“分代假说”**：

大多数对象“朝生暮死”，少数对象会活得很久。

---

4. 老生代：Major GC（标记-清除 + 标记-压缩）

目标：在不太频繁的前提下，回收长期对象空间并控制碎片。

Major GC 主要过程类似你熟悉的“标记-清除”，但带了优化：

1. **标记阶段（Mark）**
   - 从 GC Roots（全局对象、栈上的变量、闭包引用中的变量等）出发。
   - 遍历所有能到达的对象并做“已标记”记录。
2. **清除阶段（Sweep）**
   - 扫描老生代空间：
     - 已标记 → 保留
     - 未标记 → 回收
3. **压缩阶段（Compact，有时会做）**
   - 会把存活对象移动、压缩到一起，减少内存碎片，方便后续分配。

因为老生代对象通常很多、比较大，Major GC 相对 Minor GC 更“重”，

V8 会尽量减少 Major GC 的次数，或拆碎成多个小步执行。

---

5. 减少卡顿：增量 / 并行 / 空闲时间 GC

如果一次 GC 把所有工作做完，会有明显的 **“停顿（Stop-the-world）”**。

V8 做了很多优化来让卡顿不那么明显：

- **增量标记（Incremental Marking）**
  - 把“标记阶段”拆成很多小步骤，一点一点执行，
  - 中间穿插 JS 代码的正常执行。
- **并行 / 并发 GC**
  - 利用多核 CPU，让某些 GC 工作在其他线程上并行进行。
- **Idle-time GC（空闲时间 GC）**
  - 浏览器空闲时（例如用户没操作、页面空闲）多做一些 GC 工作。

你不需要记每个阶段叫什么名字，只要知道：

V8 为了减少页面卡顿，把 GC 拆散、并行、利用空闲时间执行。

---

6. 和开发者有什么关系？

你不需要会写 GC，只要知道：

- 1）只要对象是“可达”的，就不会被回收
  - 被全局变量、闭包变量、DOM 引用、单例对象等持有引用，都会一直活着。
- 2）避免无意中让对象“长期可达”
  - 不要把无用的大对象挂在 `window` / 全局单例上。

  - 事件监听器用完要解绑（尤其是绑定在 `window` / `document` / 长生命周期 DOM 上的）。
  - 定时器 / 轮询不用了要 `clearInterval` / `clearTimeout` 。
- 3）理解“短命对象没那么可怕”
  - 一些只在函数里临时创建、很快就没引用的对象，由于在新生代，很容易被 Minor GC 快速清掉。

如果你愿意，下一步我可以：

- 画一条“对象的一生”时间线（从创建 → 新生代 → 晋升老生代 → 被回收），
- 或者用一段前端实际代码（事件监听 / 闭包 / DOM 缓存）给你分析：哪些会被 V8 GC 回收，哪些可能导致内存泄漏。